

Technological Feasibility Analysis

4 April 2025

ArchaeoSight

Sponsor: Dr. Kayeleigh Sharp

Mentor: Jeevana Swaroop Kalapala

Team Members: Larry Griffith, Dazzion Porter, James Harris, Levi Ly



Table of Contents

1. Introduction	3
2. Technological Challenges	4
3. Technological Analysis	5
3.1 Machine Learning Algorithm	5
3.2 Cross-Platform Mobile Application	8
3.3 Interfacing with Client Tools	10
3.4 Data Display Scope	12
3.5 Offline and Online Support	15
4. Technology Integration	19
5. Conclusion	20

1. Introduction

Most of the land in the world has been discovered and documented. As time has gone on, people have discovered new lands and moved on from old ones. Archaeology and anthropology aim to learn about the lands they left behind as well as the people who left them. Modern archaeology uses aerial photos, LiDAR, GPR (Ground-Penetrating Radar), and texts to figure out where artifacts and important sites may be hidden. Dr. Sharp directs the Northern Gallinazo Archaeological Research Project (PIAGN) in Lambayeque Region of northern Peru. Her focus is to investigate the spatial and social contexts of certain activities, primarily copper procurement, metalworking and pottery production. She does this by collecting pXRF (x-ray fluorescence spectrometer) data, which tells her the composition of the soil. The issue with this approach is the fact that 5 mm in diameter of soil can be tested at a time.

While these modern methods allow archaeologists to collect incredibly precise data, they also introduce new technological challenges—particularly when it comes to organizing and presenting that data in a meaningful way. The limited scope of each individual reading, especially with pXRF tools, means that a large amount of complex, fine-grained data must be collected, transferred, and analyzed in tandem with GPS and image-based data. To address this, our team is building an app that not only consolidates this data but also uses machine learning to generate interactive maps and predictive models, helping archaeologists identify and prioritize promising areas for future excavation.

2. Technological Challenges

There are several major design decisions we need to make in the development of this project. Before getting into more technical details, we have a number of requirements for our project. Dr. Sharp is interested in an app that can be used across multiple platforms, most notably Android and iOS. We will need to consider how we can develop a cross-platform app as smoothly as possible. However, even the smoothest experience could be rough if we are not mindful with how we develop the app. We must have a way of passing data from the specialized tools our client and her teams use, which includes SciAps and Bruker XRF spectrometers, to our app. From our interactions with these tools, we found that they do not natively have the ability to transfer data to iOS devices, which will be a large hurdle in our development. Our app should also accept image and GPS data. Using all of this information, our app will have to use a machine learning framework to estimate potential sites for more sherds and other archaeological fragments. Early on, we will need to settle on an integrated development environment (IDE), a framework, and a programming language. Additionally, we should consider what kind of database and management system we will be using. In developing our app, we must provide users with a responsive and efficient experience to minimize frustration in inputting data. Our last technological consideration regards how internet connectivity should work for the app, as a good connection cannot be guaranteed for our users in the field. An overall issue to keep in mind with all our solutions is the limited budget of our client, meaning we should prioritize cheap or free solutions.

3. Technological Analysis

3.1 Machine Learning Algorithm

3.1.1 Introduction

One of the main focuses of this project is “An AI or ML tool that uses imagery + pXRF + GPS data for classification of sherds (and other soil/mining derived anthropogenic items) to predict GPS locations for sources of clay/material/processing.” Thankfully, there are many potential machine learning approaches for integrating these different types of data to create accurate and predictive models for archaeological site assessment.

3.1.2 Desired Characteristics

The ideal machine learning model should be able to accept multiple data types as input for training and testing, run on a mobile device as a pre-trained model, and offer results that can be understood by archaeologists.

3.1.3 Alternatives

This approach would use specialized neural networks to process images of pottery fragments and soil samples, while also incorporating location data. Deep learning is excellent at automatically finding patterns in complex data and can achieve high accuracy when properly trained. It performs exceptionally well with image data and can identify subtle visual patterns. However, this approach needs large amounts of labeled training data, which may not be available for archaeological sites. Deep learning models also work like "black boxes," making it difficult for archaeologists to understand why certain predictions are made. These models require significant computing power, which presents challenges for field use on mobile devices.

Support Vector Machines represent a more traditional approach that could handle the various types of data with some customization. This method works well for classifying data and

can efficiently handle many input variables. They use less memory than some alternatives and generally resist making overly complex models that don't generalize well. However, they struggle to effectively combine different types of data like chemical readings, images, and location information. Their decision-making process is also difficult to translate into terms meaningful to archaeologists, making results harder to validate with domain knowledge.

Random Forest combines many decision trees to make predictions based on majority vote. This approach gives reasonable results across many types of problems while providing some insight into which factors are most important. These models work well with minimal fine-tuning and can handle mixed data types reasonably well. They're also less likely to create overly specific models that don't work well with new data. However, Random Forests don't handle spatial relationships in data as effectively as needed for archaeological site prediction. They also can't identify complex interactions between different factors as well as more advanced methods.

Gradient Boosting builds decision trees in sequence, with each new tree focusing on correcting mistakes made by previous trees. This approach works exceptionally well with structured data while remaining understandable. These models excel at handling different types of data simultaneously and provide clear rankings of which factors are most important for predictions. They perform very well even with limited training examples, which is valuable when working with archaeological data where labeled examples are scarce. They also provide reliable confidence scores for predictions and can run efficiently on mobile devices, making them suitable for field use.

3.1.4 Analysis

To pick the proper algorithm, we need the one that will allow us to use multiple kinds of data, have good performance on mobile devices, be interpretable by archaeologists, and most

importantly, give us the best answer based on our data. When we compare them against each other, we can see a clear winner:

Requirement	Deep Learning	SVM	Random Forest	GBDT
Data Integration	⚠ Moderate	✗ Limited	✓ Good	✓ Excellent
Mobile Performant	✗ Limited	✓ Good	✓ Good	✓ Excellent
Uncertainty Quantification	⚠ Requires add-ons	⚠ Moderate	✓ Built-in	✓ Built-in
Interpretability	✗ Poor	⚠ Limited	✓ Good	✓ Excellent
Offline Capability	⚠ Challenging	✓ Good	✓ Good	✓ Good

3.1.5 Chosen Approach

Based on our analysis, we recommend a gradient boosting decision tree approach like XGBoost or LightGBM as our primary machine learning framework. We are choosing this one because GBDTs are excellent at combining different data types, it works well with limited examples, it provides reliable confidence levels, it is an efficient algorithm, and it can deliver understandable results.

3.1.6 Proving Feasibility

To validate our chosen machine learning approach, we propose a testing methodology that includes data preparation testing to ensure useful information can be extracted from sample pXRF readings, GPS coordinates, and archaeological images; simple model training to verify a basic GBDT model's performance on archaeological prediction tasks using a small labeled

dataset; mobile testing to convert and evaluate the model within a Flutter application prototype; accuracy assessment to compare predictions against known archaeological sites and have experts assess the model's explanations for plausibility; and comparison testing against at least one alternative approach to evaluate performance, resource usage, and prediction quality. Success will be determined by prediction accuracy, mobile efficiency, and expert validation of the model's alignment with archaeological knowledge.

3.2 Cross-Platform Mobile Application

3.2.1 Introduction

The project requires a cross-platform mobile-application that enables archaeologists to collect, process, and analyze data from multiple sources, including pXRF readings, GPS coordinates, and images. The app must also include a pre-trained machine learning (ML) model that can predict areas of archaeological activity. One of the major challenges is to ensure that the application works seamlessly across multiple devices (phones, tablets) from multiple manufacturers (Apple, Android). Additionally, the ML algorithm needs to give quick and accurate responses.

3.2.2 Desired Characteristics

The application must be able to be accessible and consistent from both iOS and Android devices, give a quick and accurate analysis of the data given to the pre-trained ML model, and be maintainable and scalable for the future. Luckily, there are two great options that fulfill these requirements.

3.2.3 Alternatives

The first of which is React Native. React Native uses JavaScript and React to build multi-platform mobile applications. It provides near-native performance using a JavaScript bridge, allowing it to work on iOS and Android. Importantly, it has a massive developer community and extensive third-party library support.

Another option is Flutter. Flutter uses the Dart programming language which compiles directly to native ARM machine code, which gives it a huge performance advantage. Dart is “write once, use everywhere”, meaning it can seamlessly work on iOS and Android devices. It has a large developer community and is backed by Google.

3.2.4 Analysis

To pick the right one, we need to look at what each framework offers in relation to our needs.

Requirement	Flutter	React Native
Cross-Platform Capability	✓ Yes	✓ Yes
Performance and Responsiveness	✓ High	⚠ Moderate
Ease of Development/Use	✓ High	✓ High
ML Integration	✓ Strong Support	⚠ Moderate
Open-Source	✓ Free	✓ Free
UI/UX Consistency	✓ Excellent	⚠ Dependent on Native Modules
Ecosystem	✓ Strong	✓ Strong

3.2.5 Chosen approach

We are going to choose Flutter as our app building toolkit for a few reasons. The first of which is performance. Because Dart compiles directly to native ARM machine code, it will provide overall better performance over JavaScript, which must implement a bridge to the environment it is working in. The second reason we are choosing Flutter is its machine learning

integration. Dart allows us to use TensorFlow Lite within mobile applications to make the ML algorithm development easier. It also has access to `flutter_map` and `google_maps_flutter` for GIS and GPS integration. The third reason we chose Flutter is because of the UI/UX consistency. React Native requires different modules for iOS and Android, whereas Dart does not.

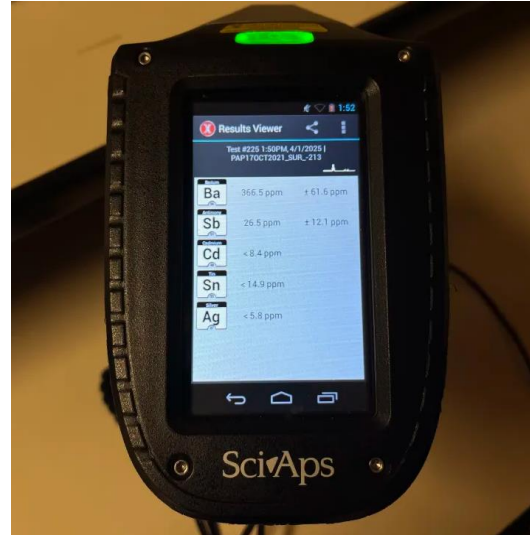
3.2.6 Proving Feasibility

To validate the choice of Flutter for the cross-platform archaeological analysis application, we propose a series of tests and demos. First, we will develop a basic prototype that integrates GPS data collection and image capturing to assess the app's core functionality. Next, we will implement a basic TensorFlow Lite model within Flutter to evaluate its ability to handle real-time data processing for machine learning tasks. Additionally, we will integrate a simple GIS visualization component to test compatibility with spatial data and ensure smooth handling of geographic information.

3.3 Interfacing with Client Tools

3.3.1 Introduction

To give informed suggestions of locations of high anthropogenic activity, it is important to first understand where our data is coming from and how it is collected. Our client Dr. Sharp plans on using ArchaeoSight in the field, in conjunction with pXRF analysis tools such as the SciAps X-200, Bruker Titan 800, and Bruker Tracer 3. These are all tools for safe environmental soil analysis, capable of precise alloy identification using X-ray fluorescence technology. By combining the geographic visualization of ArchaeoSight and the precise elemental identification of the XRF tools, Dr. Sharp will be able to conduct thorough field studies and will be able to efficiently identify anthropogenic hotspots.



3.3.2 Desired Characteristics

To ensure that our ML algorithm is making accurate predictions for areas of high-level geochemical production, one of our main objectives is to streamline spatial data with the information found by the XRF tools. This integration will enhance the algorithm's ability to identify key geochemical patterns, allowing for a deeper understanding of the relationship between elemental concentrations and its possible distribution. The given precise field data and validation of results against field observations should improve the reliability of the algorithm's predictions.

3.3.3 Alternatives

The pXRF tools capture elemental composition data in real time, recording the concentrations of the elements detected. That data is then stored, formatted, and then exported as CSV, PDF, and PDZ files; offering connectivity options through Wi-Fi, Bluetooth, or USB for file transfer. The versatility of these file formats could allow for seamless automated data transfer.

3.3.4 Analysis

Depending on which pXRF tools are used, the collected data can be formatted into a few different options. Exporting CSV files could prove to be advantageous, due to its seemingly

universal compatibility and is supported by most data analysis software. It's flexible, allowing for the ML algorithm to easily harvest data; however, depending on how the data is structured, it could be a challenge to interpret. The option of exporting data as a PDF could be easier for readability but is not very suitable for integration into software for further analysis. The PDZ file option holds rich data and is optimized for use of the manufacturer's proprietary software. This option would be perfect for our intended use but has very limited compatibility as it requires very specific software to be properly utilized.

3.3.5 Chosen Approach

With the current information given, the approach with the best outlook is exporting the data found by the pXRF tools as CSV files, as the data will be easy to clean, read, and manipulate. The transfer of these CSV files to a local PC flows seamlessly, although it is still yet to be tested if data can be exported to Android and Apple mobile devices.

3.3.6 Proving Feasibility

As stated before, further interaction with the pXRF tools is needed before understanding how data will be exported into Android and Apple systems. Exporting data as CSV files could prove to be the most convenient and available option, allowing for easy integration into ML software.

3.4 Data Display Scope

3.4.1 Introduction

One of the primary technological challenges in our project is determining how best to visualize incoming data within the app interface. Since our system aggregates high-resolution 3D scans, pXRF readings, and GPS coordinates to assist archaeologists in identifying potential areas

of interest, we must carefully design how this information is presented. Rather than opening raw files directly, our application will provide simplified, in-app visual representations of both the collected input data and the machine learning output. This decision balances usability in the field with development feasibility and device performance constraints.

3.4.2 Desired Characteristics

When evaluating how to display data, several considerations shape our approach. Creating detailed, high-fidelity visualizations for every data stream would significantly increase development time and require more UI and backend infrastructure. At the same time, archaeologists working in the field need quick, intuitive access to relevant information—especially system-generated predictions—rather than extensive, full-detail representations of every data input. By keeping raw input data in a lightweight, visual summary format, we also improve the maintainability of the app, reducing the cost and complexity of future updates. Lastly, we must ensure the interface performs well on tablets and smartphones in remote settings, where processing power and internet access may be limited.

3.4.3 Alternatives

In considering how to implement these displays, we evaluated two primary strategies. The first approach would involve a full visualization model, in which all input data—such as 3D models, pXRF scans, and spatial information—is presented in highly detailed and polished UI components. The second approach is a hybrid model in which machine learning predictions and other system-generated insights are prioritized and displayed in a clear, structured format, while raw input data is represented through simplified previews such as static images, icons, or brief text summaries. These previews are integrated directly into the app interface, allowing users to access useful information without needing to open separate files.

This hybrid model aligns with the design of many lightweight academic field tools, which emphasize usability and clarity in mobile environments. By contrast, full visualization systems, like those used in commercial platforms such as ArcGIS Field Maps, often require more powerful hardware and complex rendering engines that are impractical for field use in archaeology.

3.4.4 Analysis

Across performance, usability, and development efficiency, the hybrid model demonstrates clear advantages. It places minimal load on mobile devices, speeds up development by reducing UI complexity, and focuses on delivering the information archaeologists most need: visually guided predictions and context, not exhaustive raw datasets. This format also allows for greater flexibility in updating the system over time, since the underlying data structures can evolve without requiring major overhauls of the user interface. Importantly, the hybrid model is more compatible with the conditions and equipment commonly used in archaeological fieldwork.

3.4.5 Chosen Approach

While we have not yet made a final decision, the hybrid approach currently appears to offer the best balance between visual clarity, performance, and development feasibility. By providing in-app visual representations of both input data and ML outputs, we avoid the need for users to open external files, while still maintaining access to meaningful data. We plan to engage further with our client to ensure that this approach meets their expectations and that the simplified visual summaries still offer sufficient context and detail for their fieldwork.

3.4.6 Proving Feasibility

To validate our chosen design direction, we intend to develop early prototypes that represent both the full visualization model and the hybrid layout. These prototypes will be tested on actual mobile devices to assess performance and usability in real-world conditions. We will

also conduct feedback sessions with archaeologists and our client to determine which display strategy best supports their workflows and data interpretation needs. This hands-on evaluation will help confirm the viability of the hybrid model and ensure the application delivers useful, accessible, and field-ready information.

3.5 Offline and Online Support

3.5.1 Introduction

Archaeological sites are not typically equipped with good, stable internet. As such, an app that requires a constant connection to the internet will be inefficient at best and unusable at worst. However, a completely offline app will not meet our needs either, as our app should be able to send and receive data from a database.

3.5.2 Desired Characteristics

Our final solution should allow our app to function offline and take advantage of an internet connection while online. Offline, our app can accept and store data locally, and it may identify potential dig sites based on the local data. Online, the app can send data to and download data from a database. It should also be able to make estimates using both the local and online data. Additionally, the device going from online to offline or vice versa should minimally interrupt the user's experience; the app should not have to refresh or reset. This type of app is often referred to as an "offline-first" app.

3.5.3 Alternatives

There are three main parts of an offline-first app, and each has different potential approaches that can be taken.

Reading Data

The first is how data is read. It is important to the user that data can be accessed as quickly as possible without relying on the internet. One approach is using local data as a fallback for when the user is offline. If the app fails to obtain a piece of data from the remote API, then it instead returns locally stored data. A second approach is to use a stream, which first emits locally stored data to the user before attempting a network call. Only when the data is not found locally is a network call made. A final potential approach is using only local data. The app will simply sync with the database either manually or whenever the device is online, and the app will only use the downloaded data.

Writing Data

The second is how data is written. The first approach would be to only allow writing data when online. The next approach fully embraces the offline-first architecture, where it first stores data locally before attempting to send it to the API service.

Synchronizing Data

Finally, we consider how data is synchronized. We could approach this by simply creating a timer that runs every 10 minutes, for example, and attempts to sync the data. This could also be made to run in the background, which avoids requiring the app to be open. An alternative approach would be to utilize a push service. The server would notify the app when any data has changed, rather than having a synchronization task running in the background.

3.5.4 Analysis

Because these are quite complex to test, we did not write code ourselves to experiment. Instead, we researched each method extensively to determine which approach for each layer best suited our needs.

3.5.5 Chosen Approach

Reading Data

For reading data, having local data as a fallback is a simple approach, but attempting to make a network call first would take some time before resorting to local data. Using a stream is generally faster and less prone to error, though it is more complex to implement. Finally, relying only on local data would be fastest to retrieve data and be easiest to achieve, but it would require storing large amounts of data locally, which would likely be inconvenient for most users.

Requirement	Local Data Fallback	Stream	Local Data Only
Ease of Development	✓ High	⚠ Moderate	✓ High
Robustness (Error Resilience)	⚠ Moderate	✓ High	✗ Relies on one source of data
Speed	✗ Always makes network call	⚠ Quick for local data	✓ High, given local data
Device Impact	✓ Low	✓ Low	✗ Heavy on storage

With these factors weighed, we will opt for using a stream. Despite its higher complexity, we believe it will be best for handling the large amounts of data our app will require.

Writing Data

For writing data, online-only writing ensures that all data is fully synchronized. While users will still be able to read data offline, they will be unable to write any data. This is not suitable for our needs, as our users will often lack an internet connection while out in the field, which is where data will usually be written. The second approach allows writing data while offline, letting it be stored locally. The drawback to this is that if the app is unable to send the data to the API service, the local database and the API service will be out of sync.

Requirement	Online-Only	Offline-First
Robustness (Error Resilience)	✓ High	⚠ Synchronization conflicts may arise

Flexibility	✗ Inhibits data collection in the field	✓ High
-------------	---	--------

For our purposes, offline-first writing is the only option that fits our requirements.

Disallowing writing without internet defeats most of the purpose of our app.

Synchronizing Data

For syncing data, a timer would be simple to implement, but it can be quite taxing on the device, degrading performance and reducing battery life.

Unnecessary battery draining is especially hindering for our users, as they may not have convenient access to chargers for their devices in the field. In addition, some devices outright limit the number and power draw of processes that can run in the background, creating an additional obstacle that we will have to work around. On the other hand, pushing data from a server would only notify the app when data needs to be updated, avoiding the need for a background process. This approach will likely require additional services such as Firebase for messaging, increasing the complexity of the syncing process.

Requirement	On a Timer	Pushing Server Data
Ease of Development	✓ High	⚠ Requires additional libraries and tools
Reliability	⚠ Depends on sync interval	✓ Syncs as soon as possible
Device Impact	✗ Strong impact on performance and battery	✓ No background process required

After looking into our options, we have decided that pushing server data will be a more reasonable approach. The impact on the device from syncing on a timer is too much for our users and is the biggest factor in our decision.

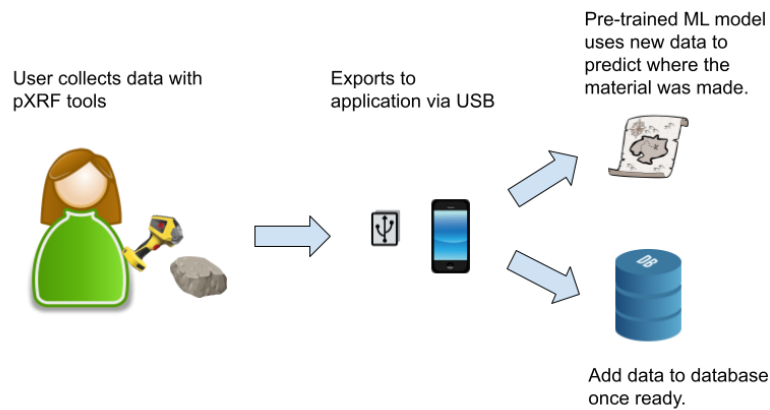
3.5.6 Proving Feasibility

Once we become adapted to our chosen framework, we plan to develop a basic prototype that combines and utilizes all three chosen approaches for reading, writing, and synchronizing simple data. We will test this prototype both online and offline to ensure that it works as expected.

4. Technology Integration

The ArchaeoSight system integrates our selected technologies into a cohesive workflow designed for archaeological fieldwork. Built using Flutter for cross-platform compatibility, the app receives pXRF data from SciAps and Bruker devices in CSV format through Bluetooth or USB connections. This data, along with GPS coordinates and site images, is processed locally by our machine learning model - a Gradient Boosted Decision Tree (LightGBM/XGBoost) implementation converted to TensorFlow Lite for mobile deployment. The model's predictions of potential archaeological hotspots are displayed through interactive maps using `flutter_map` and `google_maps_flutter` libraries.

For offline operation, the app employs local storage to maintain data accessibility in remote field conditions. When internet connectivity becomes available, the system synchronizes collected data with a central database using a push-based approach to minimize battery and performance impacts. The UI presents ML-generated predictions as priority information while providing archaeologists with simplified visual summaries of the underlying pXRF, image, and location data.



This integration of Flutter's development framework, TensorFlow Lite's efficient ML processing, and optimized data handling creates a field-ready tool that bridges the gap between precise scientific instrumentation and practical archaeological decision-making. The system maintains functionality across varying connectivity conditions while providing clear visualizations of both input data and analytical results.

5. Conclusion

In modern archaeological practices, precise and cutting-edge technology such as the pXRF tools are used. But these tools on their own call for manual data recording which can make fieldwork a bit tedious for the client. The purpose of ArchaeoSight is to uptake the heavy lifting and do the guesswork for you by creating a platform that streamlines and visualizes data, allows

for possible collaboration, and uses ML for making accurate and reliable predictions of anthropogenic hotspots.

The convenience lies within the projected compatibility of ArchaeoSight, set to be available for Android and iOS devices by being developed through the Flutter software development kit. ArchaeoSight also offers portability with its intended offline capabilities; data will be written offline first, allowing for the recording of data in the absence of an internet connection. When the user reaches a stable internet, ArchaeoSight will take advantage of this by pushing server data, as it provides quick synchronization of data without heavy strain on the end user's device and battery life. A stream will be used to allow for easy accessibility of the collected data. With data collection being built around the use of pXRF tools, this application should be easy to integrate into the typical workflow of our client at any dig site with the use of an appropriate USB connection. Our aim is to create a seamless UI fitted for archaeologists around the world, with a focus on hotspot predictions by ML algorithms, and the accessibility of clear and raw data.

These innovations together not only enhance efficiency but also reduces the workload for archaeologists, allowing them to concentrate on interpretation and discovery rather than manual data management. By bridging AI technology with archaeological field expertise, ArchaeoSight aims to be an indispensable tool in the future of Archaeology.